

Práctica 3: Minimización de un DFA

Algoritmo de refinamiento de particiones de Hopcroft

1 Objetivo

Implementar el algoritmo de minimización de autómatas finitos deterministas (DFA) en C o C++ para obtener la representación canónica con el menor número de estados posible que reconozca el mismo lenguaje regular. Comprender el impacto teórico y computacional de eliminar estados inalcanzables y fusionar estados equivalentes (redundantes), analizando cómo estas simplificaciones optimizan el uso de memoria, reducen el espacio de búsqueda en las transiciones y garantizan la unicidad estructural del autómata mínimo.

2 Algoritmo de minimización de un DFA

Algorithm 1 Refinamiento de particiones de Hopcroft

```
1: /* Input: DFA  $M = (Q, \Sigma, \delta, q_0, F)$  without unreachable states */
2: /* Output: Minimal DFA  $M' = (P, \Sigma, \delta', [q_0], F')$  */
3: Set  $P = \{F, Q \setminus F\}$ ;
4: Queue  $W = \emptyset$ ;
5: if  $F \neq \emptyset$  and  $F \neq Q$  then
6:    $W.\text{push}(F)$ ;
7:    $W.\text{push}(Q \setminus F)$ ;
8: end if
9: while  $W$  is not empty do
10:   Set  $A = W.\text{pop}()$ ;
11:   for each  $c \in \Sigma$  do
12:     /* Compute states that transition into  $A$  on character  $c$  */
13:     Set  $X = \{q \in Q \mid \delta(q, c) \in A\}$ ;
14:     for each  $Y \in P$  such that  $Y \cap X \neq \emptyset$  and  $Y \setminus X \neq \emptyset$  do
15:       Set  $Y_1 = Y \cap X$ ;
16:       Set  $Y_2 = Y \setminus X$ ;
17:       /* Replace  $Y$  with its partition  $\{Y_1, Y_2\}$  in  $P$  */
18:        $P = (P \setminus \{Y\}) \cup \{Y_1, Y_2\}$ ;
19:       if  $Y \in W$  then
20:          $W.\text{replace}(Y, \{Y_1, Y_2\})$ ;
21:       else
22:         if  $|Y_1| \leq |Y_2|$  then
23:            $W.\text{push}(Y_1)$ ;
24:         else
25:            $W.\text{push}(Y_2)$ ;
26:         end if
```

```

27:     end if
28:   end for
29: end for
30: end while
31: return P;

```

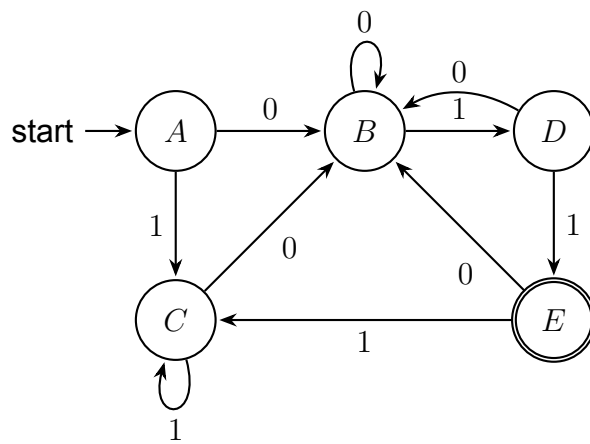
3 Ejemplo de aplicación del algoritmo

Tenemos un DFA con estados $Q = \{A, B, C, D, E\}$, alfabeto $\Sigma = \{0, 1\}$ y estado de aceptación $F = \{E\}$.

Grafo del DFA Original:

Tabla de Transiciones:

Estado	0	1
$\rightarrow A$	B	C
B	B	D
C	B	C
D	B	E
$*E$	B	C



3.1 Ejecución del algoritmo

El objetivo de este algoritmo (Hopcroft) es agrupar los estados que se comportan igual y separarlos cuando descubrimos que reaccionan diferente ante el mismo símbolo.

3.1.1 Inicialización

Separamos los estados en dos bloques iniciales (la partición P): los estados de aceptación (F) y los de no aceptación ($Q \setminus F$).

- **Partición actual (P):** $\{\{A, B, C, D\}, \{E\}\}$
- **Cola de trabajo (W):** Agregamos ambos grupos a la cola para usarlos como “distinguidores” (bloques que nos ayudarán a dividir otros). $W = [\{E\}, \{A, B, C, D\}]$.

3.1.2 Iteración 1: Usando el bloque $\{E\}$

Sacamos un bloque de W . Sea $\mathbf{A} = \{E\}$. Evaluamos cómo reaccionan los demás estados hacia este bloque con cada símbolo.

- **Con el símbolo $c = 0$:** ¿Qué estados van hacia $\{E\}$ al leer un 0? Ninguno. $X = \emptyset$. No hay divisiones.

- **Con el símbolo $c = 1$:** ¿Qué estados van hacia $\{E\}$ al leer un 1? Solo el estado D . Por lo tanto, **$X = \{D\}$** .
 - Buscamos en P algún bloque Y que tenga **algunos** estados dentro de X y **otros** fuera. El bloque $Y = \{A, B, C, D\}$ cumple esto.
 - Dividimos Y en dos: $Y_1 = Y \cap X = \{D\}$ y $Y_2 = Y \setminus X = \{A, B, C\}$.
 - Actualizamos la partición P : Ahora es $\{\{A, B, C\}, \{D\}, \{E\}\}$.
 - Actualizamos la cola W : Reemplazamos el grupo grande por los dos pequeños. $W = [\{A, B, C\}, \{D\}]$.

3.1.3 Iteración 2: Usando el bloque $\{D\}$

Sacamos de W el bloque **$A = \{D\}$** .

- **Con $c = 0$:** Ningún estado va a D con 0. $X = \emptyset$. Sin cambios.
- **Con $c = 1$:** Solo el estado B va a D con 1. **$X = \{B\}$** .
 - El bloque $Y = \{A, B, C\}$ (de la partición actual) intersecta con X de forma parcial.
 - Dividimos Y en: $Y_1 = \{B\}$ (los que van a D) y $Y_2 = \{A, C\}$ (los que no van).
 - Nueva partición P : $\{\{A, C\}, \{B\}, \{D\}, \{E\}\}$.
 - Actualizamos W reemplazando $\{A, B, C\}$ por las nuevas divisiones: $W = [\{A, C\}, \{B\}]$.

3.1.4 Iteración 3: Usando el bloque $\{B\}$

Sacamos **$A = \{B\}$** de la cola.

- **Con $c = 0$:** ¡Todos los estados del autómata van hacia B con 0! **$X = \{A, B, C, D, E\}$** .
 - Si revisamos los bloques de P , notamos que el bloque sospechoso $\{A, C\}$ está **totalmente** dentro de X . Como ambos estados se dirigen al mismo lugar, no hay motivo para dividirlos. $Y \setminus X = \emptyset$. No hay rupturas.
- **Con $c = 1$:** Ningún estado va a B con 1. $X = \emptyset$. Sin cambios.

3.1.5 Iteración 4: Usando el bloque $\{A, C\}$

Sacamos el último bloque **$A = \{A, C\}$** de W .

- **Con $c = 0$:** Ningún estado va hacia $\{A, C\}$ con 0. $X = \emptyset$.
- **Con $c = 1$:** Los estados A, C y E van hacia $\{A, C\}$ con 1. **$X = \{A, C, E\}$** .
 - Al comparar con la partición P , el bloque $\{A, C\}$ está completamente dentro de X , por lo que no se divide. El bloque $\{E\}$ también está completamente dentro de X , así que tampoco se divide.

¡La cola W está vacía! El algoritmo termina. Los estados equivalentes son A y C . Se fusionan en un solo estado AC .

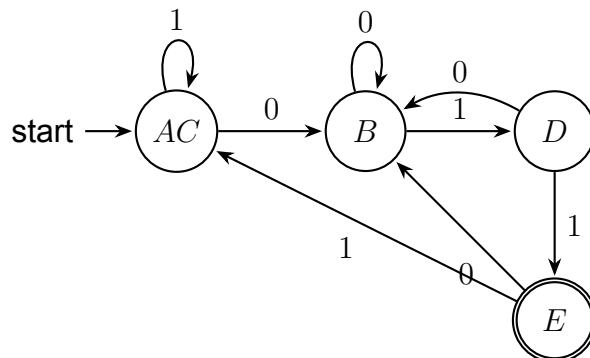
3.2 Autómata minimizado

La partición final nos dicta los nuevos estados: $Q' = \{AC, B, D, E\}$. El estado AC es el nuevo estado inicial (porque contiene al A original), y E sigue siendo el final.

Grafo del DFA Minimizado:

Nueva Tabla de Transiciones:

Estado	0	1
$\rightarrow AC$	B	AC
B	B	D
D	B	E
$*E$	B	AC



3.2.1 Análisis visual

Si observas el primer autómata, estando en el estado A o en el C , si recibías un 0, ibas a dar al estado B . Si recibías un 1, caías en un bucle hacia C . Para efectos de aceptación o rechazo de cadenas, **ser el estado A o ser el estado C daba exactamente igual**, ya que su “futuro” (sus transiciones hacia los demás) era idéntico. El algoritmo descubrió esto matemáticamente al no poder encontrar ningún símbolo que los separara.

4 Desarrollo

Para esta práctica, deberán implementar el algoritmo de minimización de DFA en C o C++. El programa recibirá como entrada una expresión regular y mostrará como salida la tabla de transiciones del autómata finito determinista antes y después de su minimización, verificando en todo momento que el número de estados inicial sea igual o mayor al del DFA minimizado ($|Q| \geq |Q'|$).

Posteriormente, seleccionarán 3 expresiones regulares de la lista provista para realizar la fase de pruebas. Para cada expresión regular elegida, deberán evaluar el autómata minimizado contra un conjunto de 20 cadenas de prueba: 10 diseñadas para ser aceptadas y 10 para ser rechazadas.

Debido a que el orden de evaluación en la construcción de subconjuntos y en el refinamiento de particiones produce identificadores de estados arbitrarios pero autómatas funcionalmente equivalentes, la evaluación no se basará en comparar nombres de estados o formatos de texto fijos mediante Docker. La validación se centrará en la corrección funcional (decisiones de aceptación y rechazo) y en la reducción correcta del número de estados. Para facilitar sus pruebas locales, pueden apoyarse en la siguiente plantilla de validación.

Lista de Expresiones Regulares para Pruebas

Elijan 3 expresiones de entre las siguientes opciones:

1. $(a|b)^*abb$ — Cadenas que terminan con el sufijo abb .
2. $(0|1)^*01(0|1)^*$ — Cadenas binarias que contienen la subcadena 01 .
3. $a(ba)^*b$ — Cadenas que inician con a , terminan con b y alternan pares internos.
4. $(a|b)^*a(a|b)$ — Cadenas sobre $\{a, b\}$ cuyo penúltimo símbolo es a .
5. $(0|10^*1)^*$ — Cadenas binarias con un número par de unos.
6. $(a|b)^*ab(a|b)^*$ — Cadenas que contienen la subcadena ab .

Descripción de Funciones de la Plantilla

La plantilla incluye las siguientes funciones clave que deben adaptar a sus estructuras de datos:

- `print_dfa(const DFA& dfa)`: Recorre e imprime la tabla de transiciones del autómata original junto con su estado inicial y su conjunto de estados de aceptación.
- `minimize_dfa(const DFA& dfa)`: Aplica el algoritmo de partición sobre el DFA inicial, descarta estados inalcanzables si existen, agrupa estados equivalentes y retorna la nueva estructura DFA.
- `print_dfa_min(const DFA& dfa_min)`: Imprime la tabla de transiciones resultante del autómata minimizado, permitiendo inspeccionar la reducción de estados frente al DFA previo.
- `test_string(const DFA& dfa, const std::string& input)`: Simula la ejecución de la cadena de entrada sobre el autómata transición por transición a partir del estado inicial; retorna `true` si al agotar la cadena el estado final pertenece al conjunto de aceptación, o `false` en caso contrario o si encuentra una transición indefinida.
- `run_test_suite(const DFA& dfa, ...)`: Itera sobre los vectores de cadenas esperadas de aceptación y rechazo, compara el veredicto de `test_string` contra el valor esperado y reporta el conteo de pruebas exitosas y fallidas.

Plantilla de Validación en C++

```
1 #include <iostream>
2 #include <vector>
3 #include <string>
4 #include <set>
5 #include <map>
6 #include <cassert>
7
8 struct DFA {
```

```

9      std::set<int> states;
10     std::set<char> alphabet;
11     int start_state;
12     std::set<int> accept_states;
13     std::map<std::pair<int, char>, int> transitions;
14 };
15
16 void print_dfa(const DFA& dfa) {
17     std::cout << "--- Tabla de Transiciones (DFA Original) ---\n";
18     std::cout << "Estado Inicial: " << dfa.start_state << "\n";
19     std::cout << "Estados de Aceptacion: ";
20     for (int f : dfa.accept_states) std::cout << f << " ";
21     std::cout << "\nTransiciones:\n";
22     for (const auto& entry : dfa.transitions) {
23         std::cout << "  d(" << entry.first.first << ", '"
24             << entry.first.second << "') -> " << entry.second << "\n"
25             ;
26     }
27 }
28
29 void print_dfa_min(const DFA& dfa_min) {
30     std::cout << "--- Tabla de Transiciones (DFA Minimizado) ---\n";
31     std::cout << "Estado Inicial: " << dfa_min.start_state << "\n";
32     std::cout << "Estados de Aceptacion: ";
33     for (int f : dfa_min.accept_states) std::cout << f << " ";
34     std::cout << "\nTransiciones:\n";
35     for (const auto& entry : dfa_min.transitions) {
36         std::cout << "  d(" << entry.first.first << ", '"
37             << entry.first.second << "') -> " << entry.second << "\n"
38             ;
39     }
40 }
41
42 DFA minimize_dfa(const DFA& dfa) {
43     DFA dfa_min;
44     // TODO: Implementar el algoritmo de refinamiento de particiones
45     return dfa_min;
46 }
47
48 bool test_string(const DFA& dfa, const std::string& input) {
49     int current = dfa.start_state;
50     for (char c : input) {
51         auto it = dfa.transitions.find({current, c});
52         if (it == dfa.transitions.end()) {
53             return false;
54         }
55         current = it->second;
56     }
57     return dfa.accept_states.count(current) > 0;
58 }
59
60 void run_test_suite(const DFA& dfa,
61     const std::vector<std::string>& accept_tests,
62     const std::vector<std::string>& reject_tests) {
63     int passed = 0;
64     int total = accept_tests.size() + reject_tests.size();
65
66     std::cout << "\n[Corriendo Casos de Aceptacion]\n";

```

```

65     for (const auto& s : accept_tests) {
66         bool res = test_string(dfa, s);
67         std::cout << "Cadena \"" << s << "\": " << (res ? "PASS" : "FAIL")
        << "\n";
68         if (res) passed++;
69     }
70
71     std::cout << "\n[Corriendo Casos de Rechazo]\n";
72     for (const auto& s : reject_tests) {
73         bool res = !test_string(dfa, s);
74         std::cout << "Cadena \"" << s << "\": " << (res ? "PASS" : "FAIL")
        << "\n";
75         if (res) passed++;
76     }
77
78     std::cout << "\nResultado: " << passed << "/" << total << " pruebas
        superadas.\n";
79 }
80
81 int main() {
82     DFA dfa_original;
83     // TODO: Construir el DFA original a partir de su pipeline de regex
84
85     print_dfa(dfa_original);
86
87     DFA dfa_min = minimize_dfa(dfa_original);
88     print_dfa_min(dfa_min);
89
90     assert(dfa_original.states.size() >= dfa_min.states.size());
91     std::cout << "\nComprobacion de estados correcta: "
92         << dfa_original.states.size() << " >= " << dfa_min.states.
            size() << "\n";
93
94     std::vector<std::string> accept_strings = {
95         // Colocar 10 cadenas que deben ser aceptadas
96     };
97
98     std::vector<std::string> reject_strings = {
99         // Colocar 10 cadenas que deben ser rechazadas
100     };
101
102     run_test_suite(dfa_min, accept_strings, reject_strings);
103
104     return 0;
105 }

```

5 Especificaciones de entrega

El reporte debe entregarse en un **único archivo PDF**. No incluyan introducciones teóricas, portadas extensas ni marcos conceptuales. Céntrense en redactar respuestas breves, razonadas y directas que demuestren que comprenden las decisiones tomadas en su código.

El documento debe contener exclusivamente las siguientes secciones:

5.1 Descripción de arquitectura y estructuras de datos

Expliquen de forma concisa cómo modelaron el autómata determinista y las particiones de estados para la minimización (por ejemplo, particiones P y conjuntos de trabajo W). Justifiquen brevemente la elección de sus estructuras (como `std::set`, `std::map` o matrices de adyacencia) considerando legibilidad, búsqueda de transiciones y manipulación de conjuntos equivalentes.

5.2 Instrucciones de compilación y ejecución de pruebas

Indiquen los comandos exactos y reproducibles para compilar su código en C o C++ y ejecutar la suite de pruebas. Incluyan evidencia puntual (capturas de terminal o salidas de texto) que demuestre:

- La tabla de transiciones del DFA antes y después de minimizar para las 3 expresiones regulares seleccionadas.
- La comprobación estricta de que $|Q| \geq |Q'|$ en cada caso.
- El resultado de evaluación de las 20 cadenas por expresión regular (10 de aceptación y 10 de rechazo) mostrando el veredicto de la suite.

5.3 Bitácora de casos límite y depuración

Describan brevemente los errores reales que enfrentaron durante la implementación del algoritmo de minimización y expliquen de manera concreta cómo los diagnosticaron y resolvieron. Por ejemplo:

- Detección y eliminación de estados inalcanzables o muertos antes de particionar.
- Manejo de transiciones incompletas hacia estados sumidero (trampa).
- Casos donde el DFA inicial ya era mínimo ($|Q| = |Q'|$).
- División errónea de particiones al calcular la preimagen $X = \{q \mid \delta(q, c) \in A\}$.

5.4 Limitaciones conocidas y deuda técnica

Señalen con honestidad qué aspectos quedaron fuera del alcance de su solución: supuestos sobre el formato de la expresión de entrada, restricciones en el tamaño del alfabeto, manejo de la cadena vacía (ϵ), o compromisos de diseño que modificarían si tuvieran que reimplementar el algoritmo de particiones desde cero.

5.5 Validación de Trabajo Presencial / Reposición Teórica

Esta sección valida el trabajo individual sobre el algoritmo y debe anexarse al final del PDF del reporte:

Para estudiantes que asistieron a la sesión presencial, incluyan una fotografía o escaneo legible del ejercicio práctico-teórico desarrollado y validado durante la dinámica en clase como comprobante de trabajo presencial.

Para estudiantes que no asistieron presencialmente a la práctica, deben resolver a mano, paso a paso y de forma exhaustiva, la **minimización de un DFA por partición de estados**. El desarrollo debe incluir:

1. La tabla de transiciones y el diagrama de estados del DFA original (no mínimo).
2. Identificación y descarte formal de estados inalcanzables (si existen).
3. Partición inicial $P_0 = \{F, Q \setminus F\}$.
4. Desarrollo iterativo paso a paso indicando el grupo divisor analizado, el símbolo de entrada evaluado, los conjuntos resultantes y las divisiones sucesivas hasta converger ($P_k = P_{k+1}$).
5. La tabla de transiciones y el diagrama de estados del DFA minimizado final.

El autómatata a minimizar se asigna según el **penúltimo dígito de su número de cuenta**:

- **Dígito 0:** Minimizar el DFA para $(a|b)^*a(a|b)$ con estados redundantes en el prefijo.
- **Dígito 1:** Minimizar el DFA para cadenas binarias que contienen la subcadena 010, partiendo de una versión con estados equivalentes en los caminos de fallo.
- **Dígito 2:** Minimizar el DFA para $(a|b)^*abb$, partiendo de un autómatata determinista de 6 estados que distinga rutas equivalentes para prefijos sin éxito.
- **Dígito 3:** Minimizar el DFA para cadenas sobre $\{0, 1\}$ con un número par de ceros y par de unos (4 estados mínimos), partiendo de una versión no mínima de 7 estados.
- **Dígito 4:** Minimizar el DFA para $a(ba)^*b$ partiendo de una representación con ramas paralelas equivalentes para la alternancia de pares.
- **Dígito 5:** Minimizar el DFA para cadenas binarias que terminan en 01 o en 10, partiendo de un autómatata con estados duplicados en la aceptación.
- **Dígito 6:** Minimizar el DFA para $(a|b)^*ab(a|b)^*$ construido con estados redundantes tras detectar la primera ocurrencia de la subcadena.
- **Dígito 7:** Minimizar el DFA para cadenas sobre $\{0, 1\}$ cuya longitud es múltiplo de 2 o de 3, partiendo de una unión de autómatatas con estados no distinguidos.
- **Dígito 8:** Minimizar el DFA para $0^*1(0|1)^*0$ partiendo de un autómatata de 6 estados con equivalencias en el lazo de estados intermedios.
- **Dígito 9:** Minimizar el DFA para $(a|b)^*a(a|b)(a|b)$ partiendo de una versión con 10 estados que contiene estados redundantes en los sufijos de longitud 2.

Nota: El ejercicio debe resolverse en papel escaneado o en tableta digital con trazo a mano alzada. No se aceptan diagramas generados por software ni texto tipografiado en computadora para este apartado. **No se debe entregar video explicando la práctica; con este ejercicio se validará su práctica.**

6 Evaluación de la Práctica

Instrucciones: Indique su grado de acuerdo con cada una de las siguientes afirmaciones utilizando una escala de 1 a 5, donde:

1. Totalmente en desacuerdo
2. En desacuerdo
3. Neutral
4. De acuerdo
5. Totalmente de acuerdo

Criterio / Afirmación	1	2	3	4	5
Los conceptos y habilidades desarrollados en esta práctica son relevantes para mi formación profesional.					
El trabajo realizado me permitió comprender de forma práctica la teoría revisada en clase.					
Las actividades y problemáticas planteadas durante la sesión resultaron atractivas y motivadoras.					
El enfoque práctico y los retos propuestos captaron mi interés a lo largo del desarrollo.					
El nivel de complejidad técnica fue adecuado respecto a los conocimientos y recursos previos disponibles.					
La claridad de la guía permitió realizar la práctica de manera fluida y sin bloqueos innecesarios.					